Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Method

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

CONTENTS

01 Method

02 Function Design

03 Parameters

04 Overloading

01

Method

Method

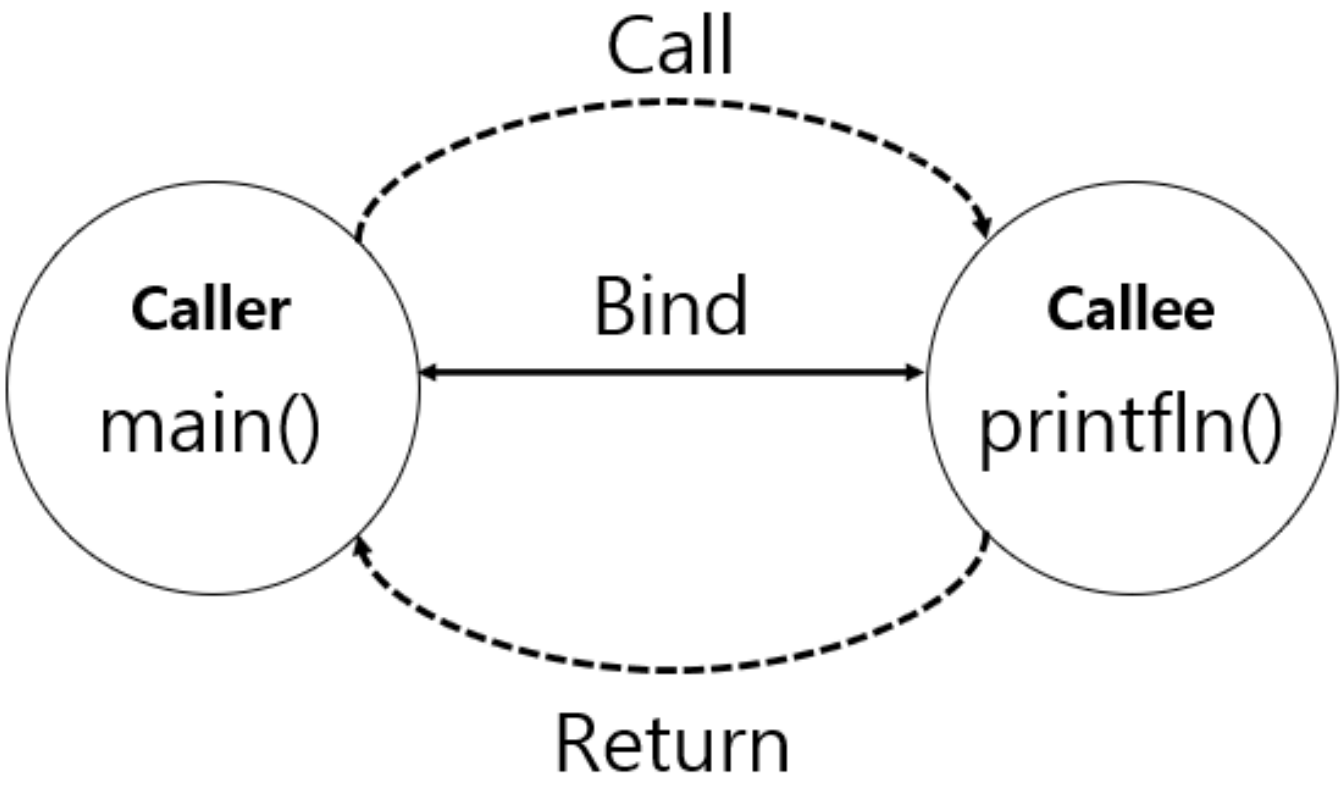
- Method (Function)
  - A unit of code that combines multiple statements with a procedural flow.
  - One of the elements that make up a class.
  - The start and end of the main() function in the Main class mark the start and end of the program.
  - When multiple methods exist, they are connected (**bind**) through calls.

Method

- Grammar Features
  - Functions are described in the form of a return type name (parameter list).
  - The relationship between **a caller function** and **a callee function** can be defined.
  - The caller is responsible for specifying **the initial parameter values** of the callee function.
  - The callee function returns a value to the caller function.
    - The return point can be changed using the return statement.
    - The function returns at the end of the function, even without a return statement.

Method

```
System.out.println("Hello");
```



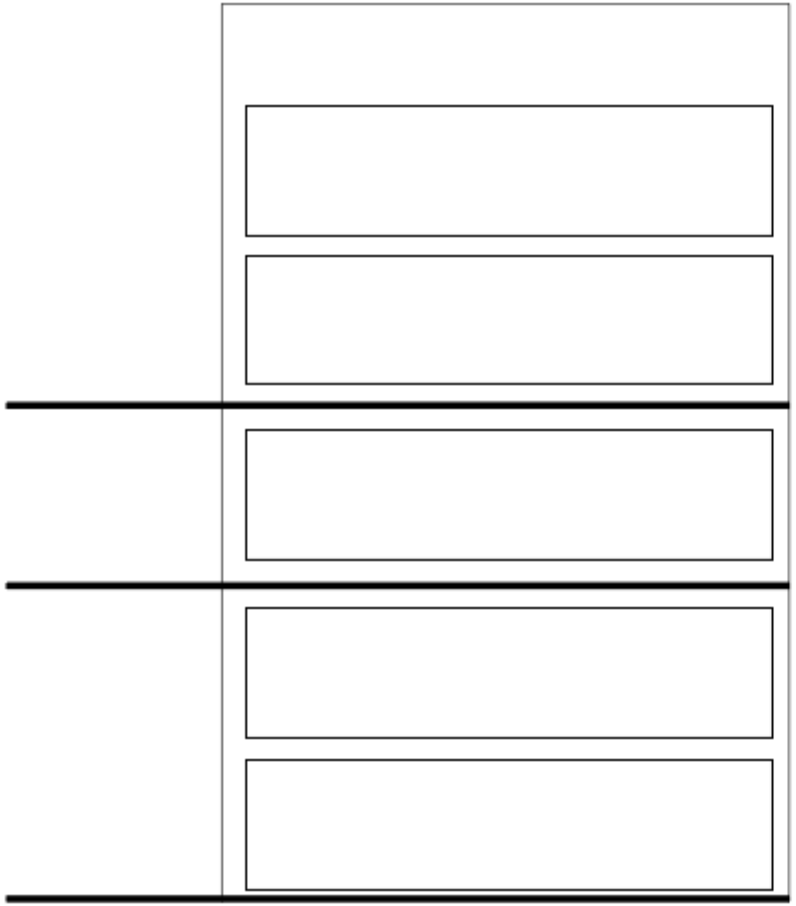
Method

- _functionSample

```
public class Main {
    public static void main(String[] args) {
        int result = add(3, 4);
        System.out.println("result: " + result);
    }

    public static int add(int a, int b) {
        int data = 0;
        data = a + b;
        return data;
    }
}
```

Method

- Function calls and stack changes



```
main()
    int a, b;
    testFunc()

testFunc(int c)
    add(1, 2)
```

Method

- Member Variables and Identifier Search Order
  - Member variables, like methods, are members of a class. They are accessible to all methods within the class and can share data with each other.
  - Identifier Search Order
    - Local Scope
    - Maximum Function Body
    - Extends to Class Members

Method

- _memberData

```
public class Main {
    static int testData = 10;

    public static void main(String[] args) {
        System.out.println("main() - begin: " + testData);
        Main.testFunc();
        System.out.println("main() - end: " + testData);
    }

    public static void testFunc() {
        int testData = 100;
        Main.testData = 300;
        System.out.println("testFunc() : " + testData);
    }
}
```

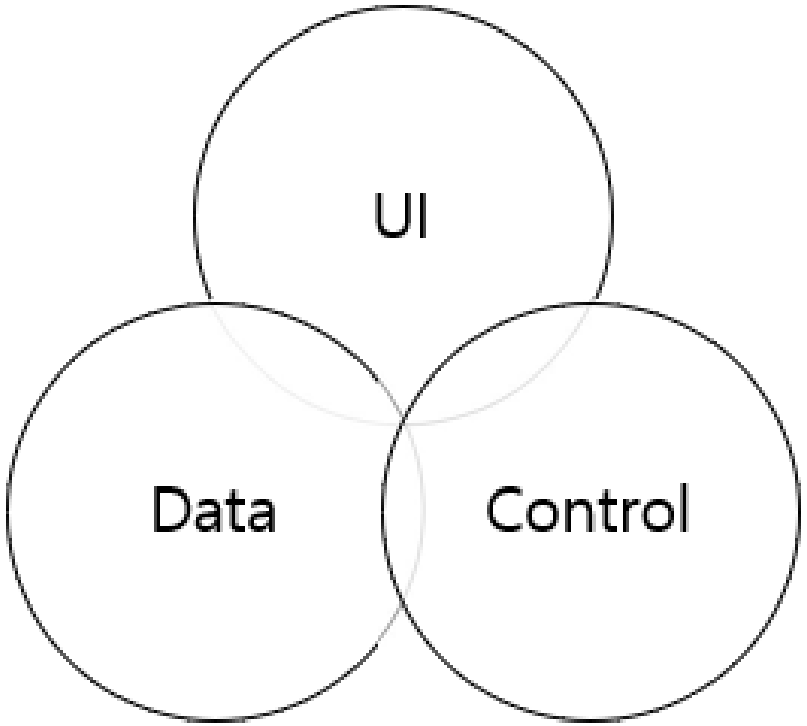
```
main() - begin: 10
testFunc() : 100
main() - end: 300
```

02

Function Design

Function Design

- Two Fundamental Functional Design Principles
  - UI and functionality must be separated.
  - Reusable units of code are implemented as functions (DRY principle).



Function Design

- _funcDesignBefore

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("factorial: "+
                getFactorial(s.nextInt()));
    }

    public static int getFactorial(int param) {
        if(param < 1 || param > 10) {
            System.out.println("ERROR: please enter the number between 1~10");
            return 0;
        }

        int result = 1;
        for (int i = 1; i <= param; i++)
            result *= i;
        return result;
    }
}
```

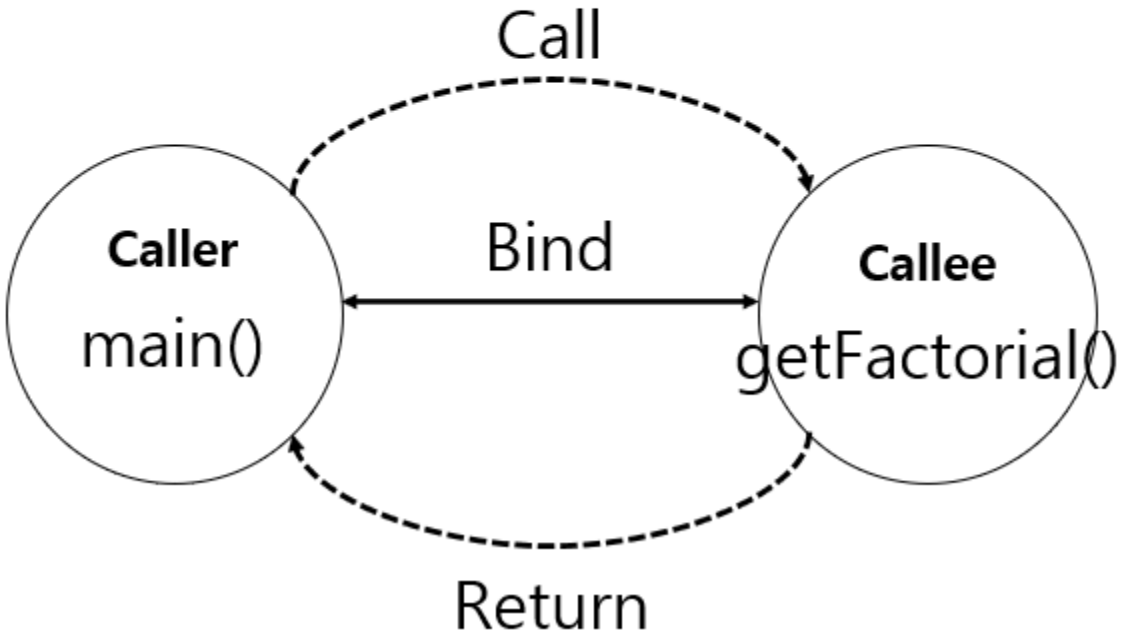
Function Design

- _funcDesignAfter

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Values for calculating factorial(1~10): ");
        int input = s.nextInt();
        if(input < 1 || input > 10) {
            System.out.println("ERROR: please enter the number between 1~10.");
            return;
        }

        System.out.println("factorial: "+ getFactorial(input));
    }

    public static int getFactorial(int param) {
        if(param < 1 || param > 10)
            return 0;

        int result = 1;
        for (int i = 1; i <= param; i++)
            result *= i;
        return result;
    }
}
```

Function Design

```
public static int getFactorial(int param) {
    if(param < 1 || param > 10)
        return 0;

    int result = 1;
    for (int i = 1; i <= param; i++)
        result *= i;
    return result;
}
```



Function Design

- _funcDesignGrade

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        int score = getScore();
        if(score < 0 )
            return;

        char ch = getGrade(score);
        System.out.println("Grade: " + ch);
    }

    public static int getScore() {
        System.out.println("Score: ");
        Scanner s = new Scanner(System.in);
        int score = s.nextInt();
        if(score < 0 || score > 100) {
            System.out.println("ERROR: The score range is
inappropriate..");
            return -1;
        }

        return score;
    }

    public static char getGrade(int score) {
        if (score >= 90) return 'A';
        else if (score >= 80) return 'B';
        else if (score >= 70) return 'C';
        else if (score >= 60) return 'D';
        return 'F';
    }
}
```

Function Design

- Event Loop
  - A structure that iterates through user interface output and user input in the main() function.
  - Typically, it displays menus and checks user selections.
  - It performs functions based on menu selections.
  - A common structure adopted by most applications.

Function Design

- _eventLoop

```
public static void main(String[] args) {
    int input = 0;

    for(printMenu(); (input = getCmdNumber()) != 0; printMenu())
    {
        if(input == 1) {
            System.out.println("New menu");
        }
        else if(input == 2) {
            System.out.println("Search menu");
        }
    }
    System.out.println("Bye");
}
```

Function Design

```
public static void printMenu() {
    System.out.println("[1]New\t[2]Search\t[3]Print\t[0]Exit");
    System.out.print("Command(0~3): ");
}

public static int getCmdNumber() {
    Scanner s = new Scanner(System.in);
    int cmd = s.nextInt();
    return cmd;
}
```

03

Parameters

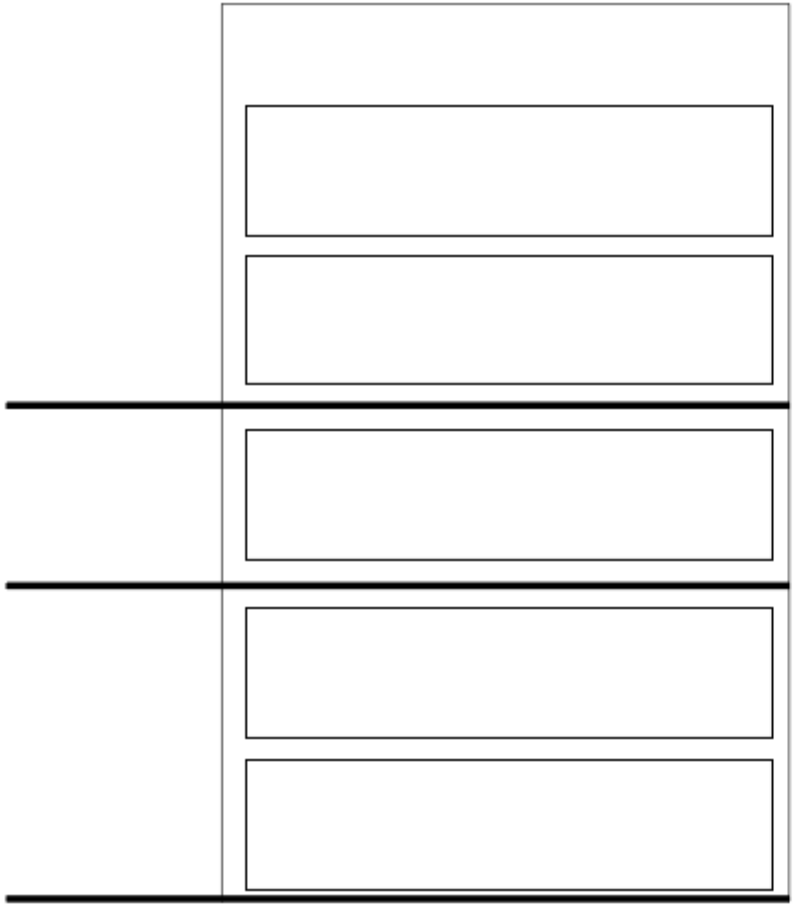
Parameters

- Function call:
  - **Call by value**
  - **Call by reference**
    - In C, reference types are implemented as pointers.

- Parameters are local variables that use the **stack**.
- All objects created using the new operation are **heap** instances.
- In Java, Strings tend to be treated as primitives.

Parameters

- Function calls and stack changes



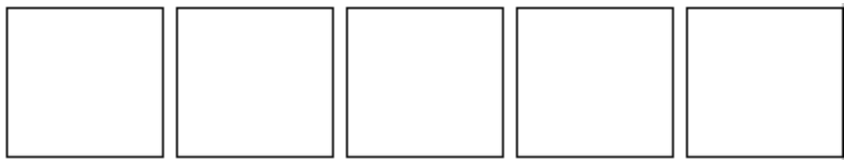
```
main()
    int a, b;
    testFunc()

testFunc(int c)
    add(1, 2)
```

Call by Value

```
public class Main {
    public static void main(String[] args) {
        int result = add(3, 4);
        System.out.println("result: " + result);
    }

    public static int add(int a, int b) {
        return a + b;
    }
}
```

Call by Reference

```
public static void main(String[] args) {
    int[] array = new int[] {10, 20, 30, 40, 50};
    printArray(array);
    modifyArray(array);
    printArray(array);
}

public static void modifyArray(int[] param) {
    for (int i = 0; i < param.length; i++) {
        param[i] = (i + 1) * 100;
    }
}
```



Recursive call

- Calling itself from within a function code
- A combination of a loop and a stack data structure
- Very important in nonlinear data structures
- Reduces **function call overhead**
- A stack overflow occurs when a logic error occurs
  - same principle as an infinite loop

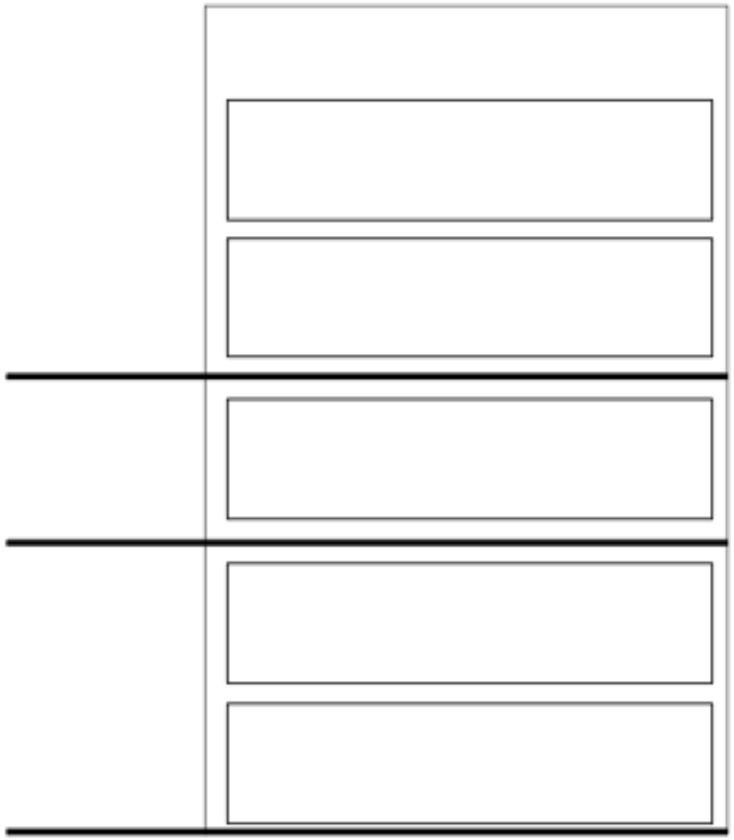
Recursive call

- _recursiveCall

```
public static void main(String[] args) {
    System.out.println(getFactorialLoop(5));
    System.out.println(getFactorial(5));
}

public static int getFactorialLoop(int param) {
    int total = 1;
    for (int i = 1; i <= param; i++)
        total *= i;
    return total;
}

public static int getFactorial(int param) {
    if(param == 1)
        return 1;

    int result = param * getFactorial(param - 1);
    return result;
}
```

```
120
120
```

Recursive call



04

Overloading

Overloading

- Methods with the same name but different parameter configurations
- Return types are not considered
- Frequently used in situations where the same functionality is provided for various datatypes

```
public static int
add(int a, int b)

public static int
add(double a, double b)
```

Overloading

```
public static void main(String[] args) {
    System.out.println(add(3, 4));
    System.out.println(add(3.3, 4.4));
    System.out.println(add("Hello", "World"));
}

public static int add(int a, int b) {
    return a + b;
}

public static double add(double a, double b) {
    return a + b;
}

public static String add(String a, String b) {
    return a + b;
}
```

```
7
7.7
HelloWorld
```

Quiz

- 1. What elements are included in a method (function) signature?
  A. Method name and access modifier
  B. Method name, return type, parameter list
  C. Parameter name and local variable list
  D. Return type and method body

Quiz

- 1. What elements are included in a method (function) signature?
  A. Method name and access modifier
  B. Method name, return type, parameter list
  C. Parameter name and local variable list
  D. Return type and method body

- Explanation
  - A method signature uniquely identifies a method. While the name and parameter list are essential, the lecture also included the return type. This serves as a criterion for determining which method to call.

Quiz

- 2. Which memory area is primarily responsible for storing local variables and parameters associated with function (method) calls during program execution?
  A. Heap Area
  B. Static Area
  C. Code Area
  D. Stack Area

Quiz

- 2. Which memory area is primarily responsible for storing local variables and parameters associated with function (method) calls during program execution?
  A. Heap Area
  B. Static Area
  C. Code Area
  D. Stack Area

- Explanation
  - The call stack is where the function execution information, along with the local variables and parameters declared within the function, are stored each time a function is called. It manages the function call and return flow.

Quiz

- 3. Java, when passing primitive type variables (such as int, double, etc.) and object type variables (such as arrays, class instances, etc.) to methods, what parameter passing method is used?
  A. Both are call-by-value.
  B. Both are call-by-reference.
  C. Primitive types are call-by-value, object types are call-by-reference.
  D. Primitive types are call-by-reference, object types are call-by-value.

Quiz

- 3. Java, when passing primitive type variables (such as int, double, etc.) and object type variables (such as arrays, class instances, etc.) to methods, what parameter passing method is used?
  A. Both are call-by-value.
  B. Both are call-by-reference.
  C. Primitive types are call-by-value, object types are call-by-reference.
  D. Primitive types are call-by-reference, object types are call-by-value.

- Explanation
  - Primitive types are call-by-value, which passes a copy of the value stored in the variable. Objects are call-by-reference, which passes a "reference" pointing to the object's location (address) in memory.

Quiz

- 4. When designing a function (method), what is the primary purpose of separating the user interface (UI) logic from the core functionality (logic)?
  A. To increase the initial execution speed of the program
  B. To minimize code duplication and the number of functions
  C. To make maintenance easier when requirements change
  D. To centralize and manage all data in a single location

Quiz

- 4. When designing a function (method), what is the primary purpose of separating the user interface (UI) logic from the core functionality (logic)?
  A. To increase the initial execution speed of the program
  B. To minimize code duplication and the number of functions
  C. To make maintenance easier when requirements change
  D. To centralize and manage all data in a single location

- Explanation
  - Separating the UI from core functionality significantly improves code maintainability by ensuring that UI changes don't affect the core logic and minimizing UI changes when core logic changes. This is also related to the DRY principle.

Quiz

- 5. What is essential to prevent a recursive function from falling into an infinite stack overflow error?
  A. A loop structure must be added within the function.
  B. Recursive calls must be made only once.
  C. State must be managed using global variables.
  D. There must be a termination condition (base case) that stops the recursive call.

Quiz

- 5. What is essential to prevent a recursive function from falling into an infinite stack overflow error?
  A. A loop structure must be added within the function.
  B. Recursive calls must be made only once.
  C. State must be managed using global variables.
  D. There must be a termination condition (base case) that stops the recursive call.

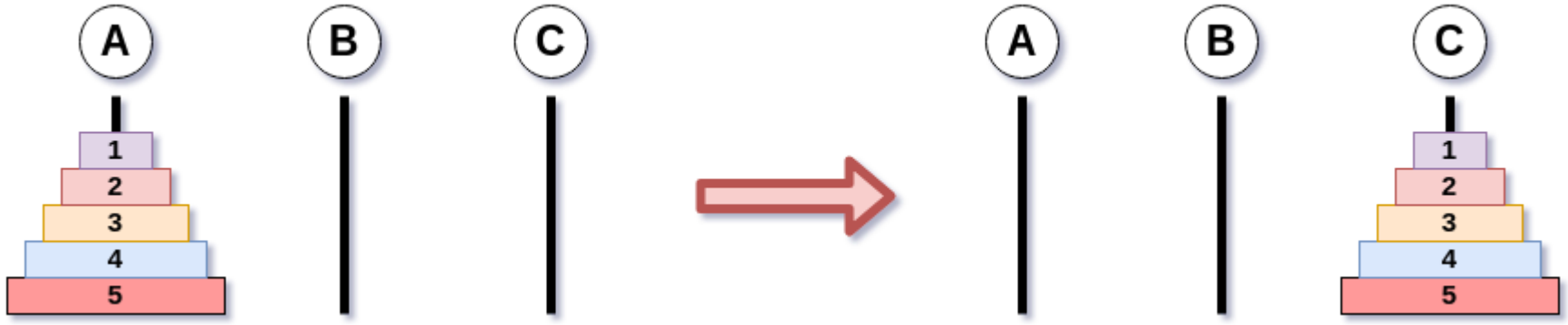
- Explanation
  - Since a recursive function continuously calls itself, it requires an "termination condition" that stops the recursive call and returns when a certain condition is reached. Without this condition, the call stack will continue to build up, eventually exceeding memory limits.

Practics

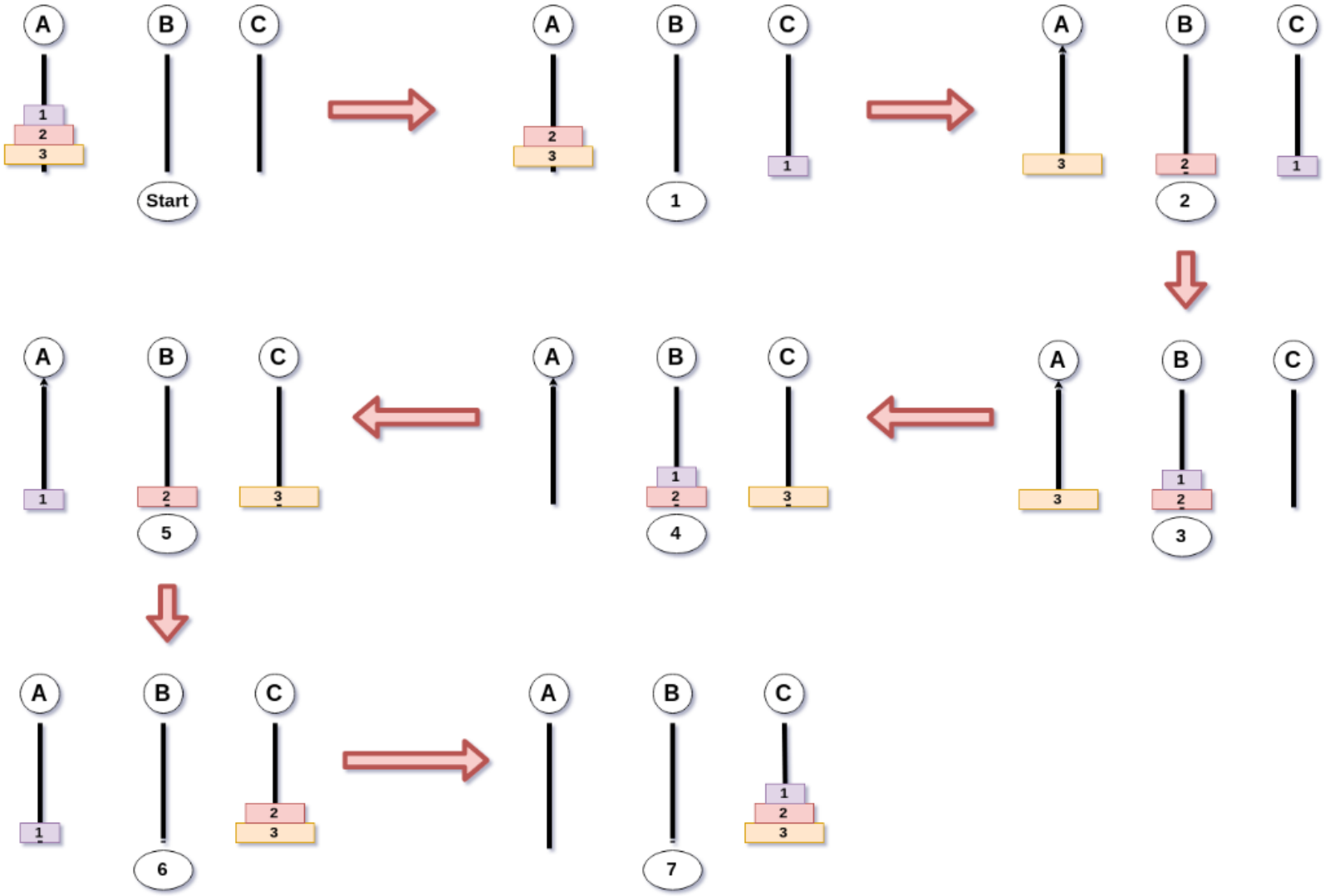
- 斐波那契数列问题
  - 斐波那契数列指的是这样一个数列 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233，377，610，987，1597，2584，4181，6765，10946，17711，28657，46368……
  - 特别指出：第0项是0，第1项是第一个1。
  - 这个数列从第三项开始，每一项都等于前两项之和。
  - 请写出程序显示前n个斐波那契数列，n由用户输入。

Practics

- 汉诺塔(Tower of Hanoi)
  - 每次只能移动一个圆盘：你一次只能从一个柱子的顶部取走一个圆盘。
  - 大盘不能压小盘：在任何时候，大圆盘都不能放在小圆盘上面。
  - 移动到其他柱子：移动的圆盘可以从一个柱子移到另一个柱子，也可以暂时放到中间的辅助柱上。
  - 目标是将所有圆盘移动：游戏的最终目标是将所有圆盘从起始柱按照大小顺序全部移到目标柱上。



Practics



Move Disk 1 from A to C.
Move Disk 2 from A to B.
Move Disk 1 from C to B.
Move Disk 3 from A to C.
Move Disk 1 from B to A.
Move Disk 2 from B to C.
Move Disk 1 from A to C.

Assignment 1

- 上吊的小人游戏 (Hangman Game)
  - 截止日期: 2025.11.20
  - 作业需要提交两份文件
    - 代码源文件(.java)
    - 运行结果展示(用pdf文档
  - 作业内容请详见作业要求文档 Assignment 1

Q & A

THANK YOU